

RealMan 真机测试操作手册

密级：内部

2026 年 7 月 2 日

目标：按照本手册逐步操作, 验证 `realman_driver`、`realman_vision`、`realman_calibration` 三个包在真机上的功能是否正常。

目录

1 准备工作	4
1.1 硬件检查	4
1.2 环境初始化	4
1.3 重要提醒	4
2 Phase 1: 基础硬件连通性	4
2.1 T1.1 —机械臂 TCP 连接	4
2.2 T1.2 —RealSense 相机采集	5
3 Phase 2: 机械臂运动与状态读取	5
3.1 T2.1 —关节位置读取	5
3.2 T2.2 —末端位姿读取	6
3.3 T2.3 —moveJ 弧度/度转换验证 (最高优先级)	7
3.4 T2.4 —急停功能	7
3.5 T2.5 —直线运动 moveL	7
4 Phase 3: 夹爪操作	8
4.1 T3.1 —夹爪基本开合	8
4.2 T3.2 —力控抓取 gripperPick	8
4.3 T3.3 —夹爪状态读取	9
5 Phase 4: 棋盘格检测验证	9
5.1 T4.1 —棋盘格检出率	9
6 Phase 5: 数据采集集成	10
6.1 T5.1 —完整采集 18 张图片	10
6.2 T5.2 —旋转多样性门控验证	11
7 Phase 6: 全流水线端到端	11
7.1 T6.1 —run_pipeline 一键标定	11
7.2 T6.2 —标定结果质量评估	12
8 Phase 7: ROS2 节点集成	12
8.1 T7.1 —calib_node 启动	12
8.2 T7.2 —服务调用	13
8.3 T7.3 —TF 坐标变换广播验证	13

9 测试汇总	14
9.1 完成情况	14
9.2 测试环境记录	15
附录：常见问题	15

1 准备工作

1.1 硬件检查

- ☐ 机械臂已开机，网线已连接到 MiniPC
- ☐ Intel RealSense D400 相机已通过 USB 3.0 连接到 MiniPC
- ☐ 棋盘格标定板已打印（ 11×8 内角点，方格边长 30 mm），贴在平整硬板上
- ☐ 机械臂工作范围内无杂物，急停按钮可触及

1.2 环境初始化

打开终端，执行以下命令：

```
# 1. 加载 ROS2 环境
source /opt/ros/humble/setup.bash
source ~/ws/realman/install/setup.bash

# 2. 确认机械臂 IP（默认 192.168.1.18，不确定问负责人）
ping 192.168.1.18      # 应该能 ping 通

# 3. 确认相机连接
ls /dev/video*         # 应能看到 realsense 相关 video 设备
```

1.3 重要提醒

单位	代码中角度用 弧度 ，长度用 米 。示教器上角度用 度 ，长度用 毫米 。对比时注意换算： $1 \text{ rad} \approx 57.3^\circ$ ， $1 \text{ m} = 1000 \text{ mm}$ 。
安全	运行运动测试前确保周围无人。急停按钮随手可及。
报错记录	任何步骤报错，截图或复制完整错误信息（包含错误码），记录到测试报告中。
测试数据	所有测试产物（图片、CSV、YAML）保留，不要删除，测试结束后统一归档。

2 Phase 1：基础硬件连通性

2.1 T1.1 —机械臂 TCP 连接

操作：

```
# 运行 gripper_test, 自动连接机械臂 (默认 IP 192.168.1.18)
ros2 run realman_driver gripper_test
```

期望看到:

```
[1] Connected to arm
[2] Gripper pre-check:
    enable: 1   online: 1   mode: ...
```

通过标准: 不报错, 能看到 “Connected to arm”; 夹爪状态 online: 1。

如果失败:

1. 检查 IP: ping 192.168.1.18
2. 检查机械臂是否开机 (示教器有显示)
3. 确认机械臂和 MiniPC 在同一网段
4. 如果 IP 不同, 用 --ip < 实际 IP> 指定

□ T1.1 通过

2.2 T1.2 —RealSense 相机采集

操作:

```
ros2 run realman_calibration collect_data \
    --ip 192.168.1.18 --output /tmp/test_cam --count 5
```

有显示器时: 弹出相机预览窗口, 能看到实时彩色画面。

无显示器时 (SSH 远程): 自动降级为 headless 模式, 终端输出状态日志。

通过标准: 连续能看到画面, 不报错; 按 q 可正常退出。

□ T1.2 通过

3 Phase 2: 机械臂运动与状态读取

3.1 T2.1 —关节位置读取

操作:

```
ros2 run realman_driver joint_test
```

连接机械臂后每秒打印一次 6 个关节角度（同时显示弧度和度），共 20 次，按 Ctrl+C 可提前退出。

期望看到：

```
[1] Joints (rad): J1=0.xxx J2=0.xxx J3=0.xxx J4=0.xxx J5=0.
    xxx J6=0.xxx
              (deg): J1=xx.x J2=xx.x J3=xx.x J4=xx.x J5=
                  xx.x J6=xx.x
```

通过标准： 6 个关节的读数均为正常弧度值（范围 $\approx \pm\pi$ ）；读数不为全零；手动轻推机械臂某关节（松开刹车后），对应读数有变化。

□ T2.1 通过

3.2 T2.2 —末端位姿读取

操作：

先用示教器记下当前机械臂末端位姿(示教器上通常显示 $x/y/z$ 毫米和 $roll/pitch/yaw$ 度)。

有显示器时，运行采集程序在 overlay 窗口读取：

```
ros2 run realman_calibration collect_data --ip 192.168.1.18 --
count 1
```

有显示器时：CV 窗口左上角显示 “Arm: x=... y=... z=... roll=... pitch=... yaw=...”（角度已换算为度）。无显示器时可改用 `joint_test`（输出含 `toolPose` 但不含此信息，建议通过有显示器的环境测试）。

通过标准： CV overlay 显示的位姿数值与示教器一致。

单位换算： 示教器角度 $\div 57.3 =$ 弧度，示教器位置 $\div 1000 =$ 米。

示例：示教器显示 $X = 500\text{ mm}$ 、 $Roll = 30^\circ \rightarrow$ overlay 显示接近 $X = 0.5\text{ m}$ 、 $Roll = 30^\circ$ （overlay 已转换为度）。

□ T2.2 通过

3.3 T2.3 —moveJ 弧度/度转换验证（最高优先级）

[WARNING] 这是最重要的测试。如果转换有 bug，机械臂会以错误的角度运动，可能造成碰撞。

背景： C SDK 内部使用度，API 对外使用弧度。Arm::Impl 在调用 SDK 前做 rad→deg 转换。movej_test 正是为此设计——发送 Joint1 = 0.5 rad 的指令，肉眼判断是否正确。

操作：

1. 在示教器上记下当前 Joint1 的角度（比如显示 10°）
2. 运行 movej_test：

```
ros2 run realman_driver movej_test
```

3. 观察机械臂实际运动后，在示教器上再次读取 Joint1 角度

通过标准： 机械臂 Joint1 实际旋转约 28.6°（即 0.5 rad），其他关节基本不动；终端输出 “MoveJ complete”，不报错。

如果 Joint1 只转了 $\approx 0.5^\circ$ 而不是 28.6°，说明弧度 → 度转换有 bug！立刻停止测试，报告负责人。

□ T2.3 通过

3.4 T2.4 —急停功能

操作：

```
# 终端 1：启动 arm_node
ros2 run realman_driver arm_node --ros-args -p arm_ip
:=192.168.1.18

# 终端 2：发送运动指令让机械臂动起来，然后立即急停
ros2 service call /arm_node/stop std_srvs/srv/Trigger "{}"
```

通过标准： 机械臂在调用 stop 后 1 秒内完全停止；不报错，服务返回 success: true。

□ T2.4 通过

3.5 T2.5 —直线运动 moveL

操作：

```
ros2 run realman_driver movel_test
```

程序连接机械臂 → 读取当前 TCP 位姿 → Z 轴 +50 mm → 执行 moveL → 读取并打印最终位姿和 Z 偏移量。

期望看到：

```
Current TCP: x=... y=... z=... roll=... pitch=... yaw=...
Target TCP:  x=... y=... z=... roll=... pitch=... yaw=...
Executing moveL (speed=20, blocking)...
Final TCP:   x=... y=... z=... roll=... pitch=... yaw=...
Delta Z = 0.0500 m (expected ~0.050)
moveL test PASSED
```

通过标准： 机械臂末端沿直线上升约 50 mm；示教器上轨迹为直线；终端 Delta Z ≈ 0.050 m。

□ T2.5 通过

4 Phase 3：夹爪操作

如果机械臂未安装夹爪，此阶段可跳过。

4.1 T3.1 —夹爪基本开合

操作：

```
ros2 run realman_driver gripper_test
```

观察各步骤：步骤 [4] 夹爪完全张开；步骤 [5] 闭合到一半（500）；步骤 [7] 完全闭合（1000）；步骤 [8] 再次张开。

通过标准： 各步骤夹爪动作正常；终端打印“All gripper tests passed.”；无超时报错。

□ T3.1 通过

4.2 T3.2 —力控抓取 gripperPick

操作：

在夹爪中间放入一个物体（如纸杯、小块泡沫），运行 `gripper_test`。

通过标准： 步骤 [6] 夹爪闭合到接触物体后自动停止（不会夹扁物体）；`gripperState` 输出 `mode` 字段显示力控模式。

□ T3.2 通过

4.3 T3.3 —夹爪状态读取

操作：

分三次运行 `gripper_test`，每次在启动前手动将夹爪调整到不同位置（全开/半开/全闭）。

通过标准： 三次的 `actpos` 值递增或递减一致；`actpos = 0` 接近全开，`actpos = 1000` 接近全闭。

□ T3.3 通过

5 Phase 4：棋盘格检测验证

5.1 T4.1 —棋盘格检出率

操作：

```
ros2 run realman_calibration collect_data \  
  --ip 192.168.1.18 --output /tmp/calib_test --count 18
```

启动后进入交互模式：

- 绿色圆点 `overlay` = 棋盘格角点被成功检测
- 红色叉号 = 未检测到
- 看到绿色角点 → 按 S 保存当前帧
- 按 Q 退出

将棋盘格放在相机视野的至少 10 个不同角度，每个角度按一次 S：

角度类型	示例
正面	棋盘格正对相机
左侧 30°	向左倾斜
右侧 30°	向右倾斜
上仰 30°	向上倾斜
下俯 30°	向下倾斜
左旋 45°	顺时针旋转
右旋 45°	逆时针旋转
近距	棋盘格靠近相机
远距	棋盘格拉远

通过标准： 10 个不同角度中 ≥ 8 个成功检出（显示绿色角点）；角点 overlay 稳定，不闪烁。

常见问题：

- 棋盘格反光 → 调整光源角度
- 棋盘格太远 → 靠近相机
- 一直检测不到 → 检查 `--board-w / --board-h` 是否与打印棋盘格内角点数一致

□ T4.1 通过

6 Phase 5：数据采集集成

6.1 T5.1 —完整采集 18 张图片

操作：

```
ros2 run realman_calibration collect_data \
  --ip 192.168.1.18 \
  --output /tmp/calib_test \
  --count 18 \
  --board-w 11 --board-h 8 --square-size 0.03
```

按照 T4.1 的方法，在 18 个不同角度各按一次 S。

注意： 机械臂每张图之间要有明显不同的旋转（不能只平移棋盘格）。

通过标准：程序结束时提示“Rotation diversity OK”(不是“WARNING: rotation diversity insufficient”);/tmp/calib_test/images/ 下有 18 张 jpg 图片;/tmp/calib_test/robot_poses.csv 有 19 行 (1 行 header + 18 行数据)。

验证命令：

```
ls /tmp/calib_test/images/ | wc -l      # 应输出 18
wc -l /tmp/calib_test/robot_poses.csv  # 应输出 19
```

□ T5.1 通过

6.2 T5.2 — 旋转多样性门控验证

操作：

重新采集一次，故意只在相似角度拍（机械臂不动，只平动棋盘格）。

通过标准：程序提示“WARNING: rotation diversity insufficient”；按 Q 可以正常退出，不会崩溃。

□ T5.2 通过

7 Phase 6：全流水线端到端

7.1 T6.1 — run_pipeline 一键标定

方式 A（推荐）：交互式全流程

```
ros2 run realman_calibration run_pipeline \
  --ip 192.168.1.18 \
  --output /tmp/calib_full \
  --count 18 \
  --mode in_hand \
  --board-w 11 --board-h 8 --square-size 0.03
```

程序依次执行：Stage 1 采集图片 → Stage 2 相机内参标定 → Stage 3 位姿处理 → Stage 4 手眼标定。Stage 1 是交互采集阶段，与 T5.1 操作相同。

方式 B（分步执行）：

```
# 第一步：采集数据
ros2 run realman_calibration collect_data \
  --ip 192.168.1.18 --output /tmp/calib_full --count 18
```

```
# 第二步：相机内参标定
ros2 run realman_calibration calibrate_camera \
  --output /tmp/calib_full --board-w 11 --board-h 8 --square-
  size 0.03

# 第三步：手眼标定
ros2 run realman_calibration compute_hand_eye \
  --output /tmp/calib_full --mode in_hand
```

通过标准：所有阶段不报错;最终产生 /tmp/calib_full/calibration_result.yaml。

□ T6.1 通过

7.2 T6.2 —标定结果质量评估

操作：

```
cat /tmp/calib_full/calibration_result.yaml
```

逐项检查：

检查项	字段	标准
旋转矩阵	rotation_matrix	3×3 , $\det \approx 1.0$ (误差 < 0.01)
平移向量	translation_vector	3×1 , 值 $< 2\text{m}$
重投影误差	reprojection_error	$< 100.0\text{px}$
条件数	condition_number	> 0 且 < 1000
求解方法	method	Tsai / Park / Horaud / Daniilidis
目测验证	观察 R 矩阵	若相机固定在夹爪附近正方向, R 应接近单位阵

辅助判断：如果 R 矩阵和单位阵差很多, 或 t 向量与相机到法兰的物理距离差很多, 可能标定有问题。

□ T6.2 通过

8 Phase 7: ROS2 节点集成

8.1 T7.1 —calib_node 启动

操作：

```
# 终端 1: 启动 calib_node
ros2 run realman_calibration calib_node --ros-args -p arm_ip
:=192.168.1.18
```

验证:

```
# 终端 2: 检查节点是否存在
ros2 node list
```

通过标准: 输出中能看到 `/calib_node`。

□ T7.1 通过

8.2 T7.2 —服务调用

操作:

在 `calib_node` 运行的状态下, 依次调用各服务:

```
# 相机标定服务 (立即返回)
ros2 service call /calib_node/calibrate std_srvs/srv/Trigger "{}"

# 手眼标定服务 (立即返回, 需先跑过 calibrate)
ros2 service call /calib_node/hand_eye std_srvs/srv/Trigger "{}"

# 紧急停止服务 (立即返回)
ros2 service call /calib_node/stop std_srvs/srv/Trigger "{}"
```

注意: `run` 服务会进入交互采集模式(与 `collect_data` 相同), 通过 `ros2 service call` 调用会一直阻塞直到人工按 Q 或 S 完成采集, 不适合作为一键验证。建议用 T6.1 的 `run_pipeline` 替代。

通过标准: 每个 `service call` 返回 `success: true`; `calibrate` 返回相机标定结果; `hand_eye` 返回手眼标定结果; `stop` 返回 `success: true` 且机械臂急停。

□ T7.2 通过

8.3 T7.3 —TF 坐标变换广播验证

操作:

在 `calib_node` 完成 `hand_eye` 标定后:

```
ros2 run tf2_ros tf2_echo base_link camera_link
```

通过标准： 能看到连续的 TF 变换输出 (Translation + Rotation)；不报 “can’t find transform”。

□ T7.3 通过

9 测试汇总

9.1 完成情况

阶段	测试项	状态	备注
P1	T1.1 —Arm TCP 连接	□	
P1	T1.2 —相机采集	□	
P2	T2.1 —关节位置读取	□	
P2	T2.2 —末端位姿读取	□	
P2	T2.3 —弧度/度转换	□	关键
P2	T2.4 —急停	□	
P2	T2.5 —直线运动 moveL	□	
P3	T3.1 —夹爪基本开合	□	
P3	T3.2 —力控抓取	□	
P3	T3.3 —夹爪状态读取	□	
P4	T4.1 —棋盘格检出率	□	
P5	T5.1 —18 张完整采集	□	
P5	T5.2 —旋转多样性门控	□	
P6	T6.1 —全流水线标定	□	
P6	T6.2 —标定结果质量	□	
P7	T7.1 —calib_node 启动	□	
P7	T7.2 —服务调用	□	
P7	T7.3 —TF 广播	□	

9.2 测试环境记录

测试日期 202__ - -

机械臂型号

机械臂 IP

相机型号

标定板规格 内角点 $\times$, 方格 mm

附录：常见问题

A. 网络不通

```
# 确认同一网段
ip addr show          # 看 MiniPC 的 IP
ping 192.168.1.18

# 确认端口 8080 开放
nc -zv 192.168.1.18 8080
```

B. 相机无法打开

```
# 检查 realsense 设备
rs-enumerate-devices

# 重新插拔 USB 线（确认是 USB 3.0 蓝色口）
```

C. 棋盘格检测不到

1. 确保光线均匀，无强烈反光
2. 棋盘格占画面 $1/4 \sim 1/2$ 面积
3. 棋盘格平整，无弯曲
4. 用 `--board-w 11 --board-h 8` 确认内角点数正确

D. headless 模式（无显示器）

SSH 远程登录时，采集程序自动降级为 headless 模式，无法用鼠标键盘交互。需要模拟按键保存帧。如果完全无交互能力，联系负责人配置远程桌面或使用 `--mode batch`

模式。